SAT DPLL solver with two-watched-literal and DLIS branching heuristics (Option 1)

Jaime Sanchez

ECE 51216

Final Project - Team 56

May 7th, 2026

Table of Contents

Table of Contents 2
1 Introduction 3
1.1 Brief Introduction 3
1.2 Programming Language 3
1.4 Code Overview 3
1.3 Running mySAT 4
1.4 Notes 4
2 DPLL and Heuristic Descriptions 5
2.1 Pseudo-code descriptions 5
2.1.1 DPLL algorithm pseudo-code 5
2.1.2 BCP with two-watched-literal pseudo-code 6
2.1.3 DLIS pseudo-code 7
2.2 Justifications for algorithm applications 7
3 Data Structures Implemented 8
3.1 DPLL 8
3.2 Two-watched-literal/BCP 8
3.4 DLIS/Branching 9
4.5 Backtracking 9
4 Testing 10
4.1 Benchmarks used 10
4.2 Tools used for validation 11
5 Results 13
5.1 Uniform Random-3-SAT benchmark results 13
6 Conclusion 17
6.1 Summarization of Results 17
6.2 Remarks 17
Appendix 18
A.1 AI Usage 18
A.1.1 Disclosure of the tools used 18
A.1.2 Description of the use 18
A.1.3 Disclosure of prompts used 19

1 Introduction

1.1 Brief Introduction

The report describes the design, implementation, and the testing methodologies processes for creating a SAT solver that uses the Davis–Putnam–Logemann–Loveland (DPLL) algorithm with the two-watched-literal and the Dynamic Largest Individual Sum (DLIS) advance heuristics. This project was completed by Jaime Sanchez who implemented the SAT solver under Option 1, as a single student team.

1.2 Programming Language

The SAT solver was chosen to be implemented in the C++ language due to Jaime Sanchez's background. Additionally C++ brings in the benefit of having faster execution time and better memory control, where a project like SAT, would benefit greatly. During the initial planning phase, it was seen that a vast majority of SAT solvers are implemented in the C/C++ language due the performance boost and that just the computation of a SAT solver would require thousands of computation calls.

1.4 Code Overview

The project was implemented under a git repository here https://github.com/sanchez-jaime/SATsolver. The repository has been set to be viewable by anyone, along with the commits made to the main branch. A README.md is provided in the github link for an overview or refer to Section 4 for more details on the data structures.

1.3 Running mySAT

Instructions for compiling and running the mySAT program can be found in the README.md file in the main directory of the project. There is a Makefile to compile the files. Use the lines given in the README.md to compile the ./mySAT executable. The program was cloned and tested in my local computer and the **eceprog2** computer via SSH.

1.4 Notes

Due to personal timing issues, the majority of the project was completed the few last weeks of the assignment deadline. In order to fully satisfy the requirements, and implement a good approach to SAT, the BCP aspect of DPLL and the two watched-literal heuristics were implemented simultaneously. Therefore there are no metrics regarding the independent testing implementations between the basic DPLL algorithm without the advanced heuristic. The DLIS portion does have, and was, implemented to be independent and details regarding the timing aspects are noted in Section 3.2 DLIS/Branching.

2 DPLL and Heuristic Descriptions

2.1 Pseudo-code descriptions

The following sections describe the overall pseudo-code implementation of DPLL, BCP (with two watched literals), and DLIS.

2.1.1 DPLL algorithm pseudo-code

```
DPLL(CNF_function, literal_watches, literal_assignments)
   if BCP(CNF_function, literal_watches, literal_assignments) == CONFLICT
      return UNSAT

   lit = DLIS(CNF_function, literal_assignments)                    # DLIS
   if lit == 0                                                      # all variables assigned
      return SAT

   temp_status= trail.size()              #save status of queue for backtracking

   queue add (lit)
   if DPLL(CNF_function, literal_watches, literal_assignments) == SAT  #recursion #1
      return SAT
   rewind_trail_to(temp_status)         #perform backtraking
   queue.clear()

   queue add (-lit)
   if DPLL(CNF_function, literal_watches, literal_assignments) == SAT  #recursion #2
      return SAT
   rewind_trail(temp_status)
   queue.clear()

   return UNSAT                                                     # no SAT assignments found
```

The DPLL function produces the core SAT solver algorithm. The function contains the BCP pre-check for finding unit cases and the doing the two-watched literal heuristics, picks the new best literal to branch to (DLIS), and performs recursion for repeating BCP and DLIS until a satisfiable solution is found. If there is no solution, then DPLL returns false for UNSAT.

2.1.2 BCP with two-watched-literal pseudo-code

```
BCP(CNF_function, literal_watches, literal_assignments)
   while queue is not empty
      lit = queue.pop_front()
      if value(lit) == TRUE
         continue                                  # already satisfied
      if value(lit) == FALSE
         queue.clear()
         return CONFLICT                           # contradiction/will need to backtrack

      assign(lit)                                  # adds to queue trail

      for each clause in watches[-lit]
         (other, falsified) = watched_literals_of(clause)
         if value(other) == TRUE
            continue                                # clause is satisfied
         new_watch = find_non_false_unwatched(clause, assignments)
         if new_watch found
            false == new_watch                    # swap and look at a new literal
         else if value(other) == UNASSIGNED
            queue.push(other)                       # forced assignment
         else
            queue.clear()
            return CONFLICT                          #conflict found, will backtract in DPLL
   return OK
```

The BCP function performs the unit case assignability along with implementing the two-watched-literal heuristic. So long as there are no contradictions while assigning literals and available unassigned literals, BCP will continue to assign literals until it is time to branch onto a new literal (taken care of in the DPLL function by DLIS).

2.1.3 DLIS pseudo-code

```
DLIS(CNF_function, literal_assignments)
   pos_count= []                                  #record keeping for all positive literal count
   neg_count= []                                  #record keeping for all negative literal count

   for each clause in CNF_function:
      if clause.is_satisfied(literal_assignments)
         continue
      for each literal in clause                  #counter for occurrences of literals that are unassigned
         if value(literal) == UNASSIGNED
            if l > 0:
                  pos_count[l]   += 1
            else:
                  neg_count[abs(l)] += 1

   best_lit = 0
   best_count = 0
   for each literal in CNF_function
      if literal == UNASSIGNED
         if pos[literal] > best_count
            best_lit =  literal
            best_count = pos_count[literal]
         if neg[literal] > best_count
            best_lit = -literal
            best_count = neg[literal]
   return best_lit                                 #found new literal to branch from using DLIS method
```

The DLIS function simply picks the new best variable to branch from. DLIS uses a counter to determine which unassigned variable, with the most frequency, should be next to be branched too.

2.2 Justifications for algorithm applications

The project focused on satisfying the requirements set by Option 1 of implementing a SAT solver with the basic DPLL and two advanced heuristics. BCP and DLIS are meant to be straight forward applications of the SAT. Both are implemented as iterative approaches, making programming such algorithms as easy as possible. The baseline DPLL algorithm uses the recursive approach to perform BCP, DLIS, and then recursively call itself to find assignables based on the polarity of the new-branching literal.

3 Data Structures Implemented

3.1 DPLL

The baseline DPLL algorithm performs a recursive approach to implementing SAT. DPLL performs the following in the exact order: BCP, DLIS, and the recursion. First DPLL handles the precondition base cases of calling BCP. BCP handles the trivial cases and the two-watched-literal heuristics. Once all clauses are exhausted within the BCP function, DPLL then calls on DLIS to pick the new best literal to branch onto. Lastly DPLL will either branch on the literal in either its current polarity or the opposite based on the specific recursion callback. The DPLL function also handles the queuing of the new literal to branch on and also performs a safe state checkpoint to revert back to, if DPLL requires to backtrack. If DPLL returns true, then the mySAT program has found a satisfiable variable assignment to the given cnf benchmark formula.

3.2 Two-watched-literal/BCP

The BCP function and algorithm performs the single most important aspect of DPLL. BCP handles the trivial cases of solving unit clauses all while performing the two-watched-literal heuristic on two literals for each clause that has not yet been satisfied. BCP first looks and pops through the queue to assign any new literals so long as they satisfy the clause. For when a new literal is assigned, BCP also performs the action of replacing the previously picked and assigned literal for the queue in order to satisfy the two-watched-literal watch list. BCP repeatedly performs these actions until a conflict is found or the queue is empty. BCP returns true for when all assigned literals within the precondition action of DPLL have been satisfied, otherwise a false would be returned indicating a conflict which would lead to backtracking so long as it is not a trivial clause.

3.4 DLIS/Branching

DLIS performs the most simple action within DPLL. The sole purpose of DLIS is to pick the new best literal for DPLL to branch on. The DLIS heuristic uses a statistical approach to figuring out what is the new best literal to branch on that gets returned to DPLL. The DLIS function does this by recording the number of times each unassigned literal appears in an unsatisfied clause. The most occurring literal that has been recorded would be returned back to DPLL to branch to next.

The mySAT program also includes a simple branching heuristics that was used during the early phases of the program. The basic_brancher() function simply chooses a new unassigned literal within all the variables in the given CNF formula. Refer to the README.md file in the main directory of the project for instructions for choosing this branching functionality.

4.5 Backtracking

The backtracking aspect of DPLL is handled using a safe state checkpoint approach by recording the size of the queue. If it is required to backtrack, DPLL would simply return back to where the size of the queue was before undoing its failed branching operation.

4 Testing

4.1 Benchmarks used

Table 1. Lists of Benchmarks

Benchmark	Vars	Clauses	Class
simple_sat	3	3	SAT
unit_clauses	3	3	SAT
mixed_sizes	4	5	SAT
shared_watches	4	4	SAT
trivial_unsat	1	2	UNSAT
pigeonhole_3_2	6	9	UNSAT
uf20/	20	91	SAT
uf50/	50	218	SAT
uf75/	75	325	SAT
uf100/	100	430	SAT
uf125/	125	538	SAT
uf150/	150	645	SAT
uf175/	175	753	SAT

uuf50/	50	218	UNSAT
uuf75/	75	325	UNSAT
uuf125/	125	538	UNSAT
uuf175/	175	753	UNSAT

As mentioned in the project overview and guidelines, the use of AI is allowed for such infrastructure code: input parsing, testing, or validation. While programming and designing the SAT solver, Clause AI was used to generate six cnf formulas that were the backbone of testing for the simple DPLL and BCP/two-watched-literal heuristic implementation. The six cnf formulas can be found in *simple_benchmarks/* directory and referenced in Table 1 (the first six). Each of the six benchmarks were generated to cover core DPLL functionality and common trivial cases to be used while debugging. Additional details regarding what each of these six benchmarks do can be found within the .cnf file.

Additionally, Uniform Random-3-SAT benchmarks, from the SATLIB - Benchmark Problems at https://www.cs.ubc.ca/~hoos/SATLIB/benchm.html, were used in the later portion of design and implementation to test the SAT solver against large number of literals and clauses. These benchmarks are located in the benchmarks/ directory of the project and referenced in Table 1.

4.2 Tools used for validation

Two shell scripts were created by Clause AI to help with running multiple benchmarks within one call. Refer to the project's directory README.md for these two scripts 'simple_test_runner.sh' and 'test_runner.sh'. Both shell scripts generate a results log to output all relevant information, regarding execution time and or results with a SAT verdict. These logs are auto generated and published in the

benchmark_logs/ directory. The ‘simple_test_runner.sh’ script runs the first six benchmarks, listed on Table 1, that was used originally to test the fundamental DPLL code. The ‘test_runner.sh’ script was used to test the first 10 benchmarks from each Uniform Random-3-SAT benchmark. In order to save time, not all of the benchmarks from each directory of the Uniform Random-3-SAT problems were tested, but enough were to get the grasp of the capability of the mySAT program.

5 Results

5.1 Uniform Random-3-SAT benchmark results

Table 2 mySAT results of the Uniform Random-3-SAT benchmark

Type	Variables	Type	Median (s)	Mean (s)	Max (s)
uf20	20	SAT	0.002	0.002	0.003
uf50	50	SAT	0.003	0.003	0.005
uf75	75	SAT	0.004	0.004	0.007
uf100	100	SAT	0.005	0.008	0.035
uf125	125	SAT	0.011	0.035	0.146
uf150	150	SAT	0.041	0.202	0.932
uf175	175	SAT	0.821	1.150	3.601
uuf50	50	UNSAT	0.003	0.003	0.004
uuf75	75	UNSAT	0.008	0.009	0.011
uuf125	125	UNSAT	0.167	0.163	0.258
uuf175	175	UNSAT	3.047	4.726	15.340

Under the ‘test_runner.sh’ script, 110 benchmarks were tested against the mySAT program using both advanced heuristics along with the baseline DPLL. Table 2 reports the runtime results, with all advanced

heuristics enabled, on the Uniform Random-3-SAT benchmarks, which include both satisfiable and unsatisfiable instances. The benchmarks tested the program against a large number of literals and clauses. These benchmarks ranged from 20-175 variables, and to 91-753 clauses.

Table 3 mySAT results of the Uniform Random-3-SAT benchmark without DLIS

Type	Variables	Type	Median (s)	Mean (s)	Max (s)
uf20	20	SAT	0.002	0.002	0.003
uf50	50	SAT	0.003	0.003	0.006
uf75	75	SAT	0.010	0.012	0.042
uf100	100	SAT	0.073	0.066	0.110
uf125	125	SAT	0.958	1.320	3.454
uf150	150	SAT	8.180	9.742	31.300
uf175	175	SAT	13.922	39.577	129.675
uuf50	50	UNSAT	0.004	0.004	0.006
uuf75	75	UNSAT	0.023	0.027	0.067
uuf125	125	UNSAT	2.901	4.060	9.118
uuf175	175	UNSAT	181.049	230.511	623.846 (10-minute timeouts)

Table 3 shows the results for when the DLIS branching heuristic is not enabled. Using the same set of benchmarks from table 2, the runs without DLIS enabled show how much the branching heuristic contributes significantly to large formula problems. Such instances can be visibly seen for the UNSAT benchmarks where the search tree of the unassigned variables must be repeatedly and blindly iterated through. Overall, DLIS does not affect the the output of the mySAT program, but rather the most

important characteristic of the heuristic is the amount of time and space saved while using a statistical heuristic approach for the DPLL branching problem.

As described in Section 1.4 Notes, the two-watched-literal heuristic is embedded within the BCP function. There is no way of inhibiting the two-wathcd-litereal heuristic without redesigning the BCP functionality. Comparison can only be made on DPLL with BCP and two-watched literal, or two-watched literal and DLIS.

6 Conclusion

6.1 Summarization of Results

The mySAT solver produced correct verdicts on all 110 benchmarks, regardless of heuristic configuration. Comparing Table 2 (DLIS enabled) against Table 3 (DLIS disabled) shows that the advanced heuristic substantially reduced execution time across nearly every type of benchmark. The verdicts themselves never changed, but the runtime data confirms that the heuristic delivers a meaningful improvement to DPLL performance.

6.2 Remarks

This project successfully implemented a complete SAT solver based on the Davis–Putnam–Logemann–Loveland (DPLL) algorithm, with the addition of the two-watched-literal and DLIS heuristics. The code and the design were done to satisfy Option 1 of the course's project requirements. If required, the program can be cloned to a local machine from the following link: https://github.com/sanchez-jaime/SATsolver. The program has been tested on both the eceprog2 machines and the local machine I developed on. Additionally, within the code, the global variable DLIS_INHIBITED can be set to true to test mySAT's capability without the DLIS heuristic. Any other relevant information can be found in the project's README.md file in the main directory.

Appendix

A.1 AI Usage

A.1.1 Disclosure of the tools used

For this project, and within the scope of the GENERATIVE AI USAGE POLICY, Claude AI was used. Claude AI used the Opus 4.7 model for the outputs. Claude was used for these specific uses: input parsing, testing, or validation.

A.1.2 Description of the use

Claude AI produced the following files: Makefile, parser.c, parser.h, simple_test_runner.sh and test_runner.sh. Each of these files are tagged appropriately in the headers to distinguish AI generated code from my own. Additionally, these files and scripts were generated within the project's AI guidelines. The files 'parser.c' and 'parser.h' serve only to parse the input files where then 'mySAT.cpp' would use the data from 'parser.c' to regenerate the input formal struct.

The testing scripts 'simple_test_runner.sh' and 'test_runner.sh' were used to streamline the testing and recording the run logs. These scripts only serve to be helpful tools that do not impact any or influence any of the decisions of the algorithms designed or implemented within 'mySAT.cpp'.

NOTE: No aspect of Claude AI was used to generate source code that led to any of the functionality of DPLL, BCP, two-watched-literal, or DLIS.

A.1.3 Disclosure of prompts used

Prompts used for parser.c and parser.h:

1. Generate parser.h and parser.cpp for a SAT solver in C++. The parser must parse a DIMACS CNF files format into a Formula struct that includes number of variables, number of clauses, and a simple data type to put each clause to make it easily accessible.
2. Make sure the parser files are able to parse benchmarks from the following website, and be universal for any other DIMACs format for a SAT solver. Here is the website: https://www.cs.ubc.ca/~hoos/SATLIB/benchm.html

Used the first implementation of a parser files.

Prompts for the six simple benchmarks:

1. write simple cnf benchmark files (in DIMAS format) to test out a two-watched-literal or a bcp implementation. These should cover the barebones trivial cases, for debugging.
2. Make the benchmarks more simple to handle unit cases
3. Make a benchmark for the most common ways a dpll sat solver would get stuck at

Prompts for the simple_test_runner.sh and test_runner.sh scripts:

1. write a bash script that runs my program with different benchmarks located in the /simple_benchmarks directory. Here is how I would execute the program manually ./mySAT simple_benchmark.cnf
2. I want all the results to be added to a directory with the timestamp within a benchmark_logs/ directory so that every time I ran the script, it will just add another directory to benchmark_logs
3. create another similar test script. This time to run multiple benchmarks within the /benchmarks directory. At the end create a single file text summary of the elapsed time it took for the sat solver to complete each individual benchmark. Have the summary also have a statement if the benchmark was proven to be sat or not
4. I still want the summary text file to be dumped in the benchmark_logs directory
5. Some of the cnf files might be nested in multiple directories within the benchmarks directory
6. now update the script to test the first 10 cnf formulas from each directory that are in /benchmarks

Prompt for the Makefile:

1. I want a simple makefile that still uses a similar compile line like “g++ mySAT.cpp parser.cpp -o mySAT” but can be handled in (c++ (Ubuntu 11.4.0-1ubuntu1~22.04.3) 11.4.0) or (c++ (Ubuntu 13.3.0-6ubuntu2~24.04.1) 13.3.0) versions.